

## Reusabilidade

### 1 - Introdução

Uma das características da Programação Orientada a Objetos é promover o reuso de software

- Reduz tempo de desenvolvimento, custo de manutenção
- Simplifica criação de novos sistemas, novas versões de sistemas antigos.

Técnicas de projeto tornam o software OO mais reusável:

**Abstração de dados** => Sistema modular => mais fácil para compreender

**Herança** => Classes compartilham métodos => Programação-por-diferença

### **Framework**

-Conjunto de classes que incorporam um projeto abstrato para soluções de uma família de problemas relacionados, e provê reuso em uma granularidade maior do que classes.

Um projeto de classes reusáveis requer julgamento, experiência e experimentação por parte dos projetistas.

### 2 - POO

**-Polimorfismo:** Operações são executadas em objetos enviando-se mensagens a esses objetos; Um envio de mensagem é implementado buscando-se o método correto na classe destinatária e invocando esse método; o envio de mensagens causa o polimorfismo.

Exemplo: um método que soma elementos em um vetor funciona se todos os elementos do vetor compreenderem a mensagem de adição, independente da classe a que pertençam.

**-Protocolo:** A especificação de um protocolo é dada por seu protocolo; os protocolos são importantes para os programadores se comunicarem. Programadores usam o mesmo nome para operações em classes diferentes; Linguagens procedurais precisam criar nomes diferentes para operações similares, para que essas operações possam ser usadas em um mesmo programa.

### **-Herança x Decomposição**

Às vezes uma subclasse seria melhor como componente; Comportamentos podem ser mais fáceis de reutilizar como um componente do que por herança.

**-Classe abstrata:** Não possui instâncias; classes no topo de hierarquias geralmente são abstratas; geralmente não define variáveis de instância (atributos); define métodos a serem implementados em subclasses;

### 3 - Reuso de Software

Desenvolver sistemas é caro, e mantê-los é ainda mais caro, Uma forma de reuso é melhorar sistemas. A manutenção é uma forma de reuso.

#### **-Manutenção de Software**

Corretiva → Diagnosticar e corrigir erros

Adaptativa → Adaptar sistema a novo hardware

Perfectiva → Melhorar desempenho e manutenibilidade

-Características da Programação Orientada a Objetos favorecem a manutenção

Modularidade → Facilita a compreensão do efeito de mudanças no sistema.

Polimorfismo → Reduz a quantidade de código a ser compreendido.

Herança → Permite criar novos códigos sem afetar os antigos.

### **Reflexão Computacional**

Atividade realizada pelo sistema computacional quando faz computação sobre a própria computação.

Definição: Em ciência da computação, reflexão computacional (ou somente reflexão) é a capacidade de um programa observar ou até mesmo modificar sua estrutura ou comportamento. Tipicamente, o termo se refere à reflexão dinâmica (em tempo de execução), embora muitas linguagens suportam reflexão estática (em tempo de compilação).

#### **-Causalmente conectado**

Um sistema está causalmente conectado a um domínio se as estruturas internas e o domínio que ele representa estão ligados de forma que se um deles muda, isso leva ao efeito correspondente de outra.

#### **-Sistema reflexivo**

Incorpora estruturas que representam o próprio sistema; auto apresentação; o sistema responde a questões e atua sobre ele mesmo; pode fazer modificações em si mesmo a partir de seu próprio processamento.

#### **-Valor prático substancial: exemplos**

Manter estatísticas de desempenho; Auto-otimização; Manter informação para depuração; Auto-modificação (aprendizagem).

#### **-Arquiteturas Reflexivas**

Interpretador da linguagem fornece ao programa em execução acesso a dados que representam o programa. O programa pode conter código que define como esses dados podem ser manipulados (código reflexivo); Modificações realizadas pelo

programa em sua própria representação são refletidas no comportamento do programa e no seu funcionamento; Um “novo” paradigma de programação.

O sistema tem uma parte de objetos [resolve problemas do domínio] e uma parte reflexiva [resolve problemas e retorna informação da parte de objetos].

Exemplo: Exibir informações de rastreamento de programas em um sistema baseado em regras.

-Com reflexão: adicionar código reflexivo

IF uma regra tem a maior prioridade na situação

THEN imprima a regra e os dados que levaram a essa condições

-Sem reflexão: adicionar um código

print(regra+dados) após CADA regra

### Benefícios da reflexão:

Melhor modularidade, o que leva a sistemas mais gerenciáveis, legíveis, fáceis de compreender e modificar.

### Arquiteturas reflexivas

Linguagens procedurais, lógicas, e baseadas em regras

Tem em comum interpretador meta-circular: uma representação da interpretação da linguagem; a representação é usada para executar (interpretar) a linguagem; ele é uma maneira fácil de garantir a conexão causal.

-Reflexão procedural: a mesma representação implementa o sistema e contém dados do sistema, a linguagem e os dados possuem formato comum;

-Reflexão declarativa: a auto-representação do sistema não é a implementação do sistema;

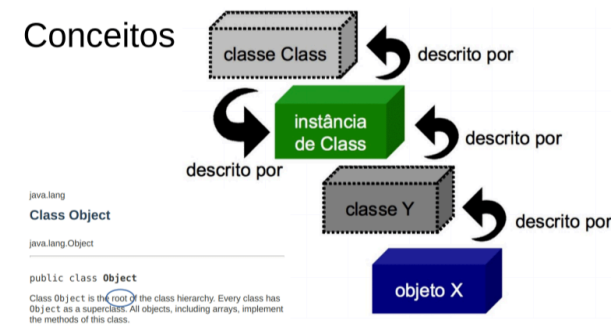
### Arquitetura reflexiva em uma linguagem orientada a objetos

- 1) Divisão ordenada entre nível de domínio e nível reflexivo
  - Cada objeto [contém dados sobre o domínio] tem um meta-objeto [contém dados de reflexão] (que aponta para o objeto).
- 2) Auto-representação uniforme: tudo é objeto
  - Instância, classe, atributo, métodos, meta-objetos, mensagens, ...
  - Todo aspecto pode ser refletido.
- 3) Meta-objetos contém toda informação disponível na linguagem => o conteúdo do meta-objeto é base do interpretador.
- 4) Consistência: a auto-representação é usada para implementar o sistema; cada ação realizada em um objeto é feita requisitando o meta-objeto; ex: criar um instância, enviar uma mensagem...
- 5) A auto-representação pode ser modificada, refletindo na computação em progresso.

**Metaprogramação** - A utilização de reflexão também é conhecida como metaprogramação, pois com sua utilização um programa pode realizar computações a respeito dele próprio.

**Anotações** - permitem marcar os elementos de forma que um algoritmo que utilize reflexão possa identificar os elementos que ele deve tratar de forma diferente

Uma classe possui meta informações a respeito de seus objetos  
A classe descreve o objeto mas quem descreve a classe?



Como obter a classe de um objeto? Via referência estática, via objeto ou via string.

## Padrões de Projeto

De forma geral, todos os padrões factory encapsulam a criação de objetos.

### Factory Method

Encapsula a criação de objetos, mas, deixando as subclasses decidirem quais objetos criar

delega a implementação do código de criação para as suas subclasses ou classes que o implementam.

Segundo o GOF (Group Of Four) o padrão Factory Method é: “Um padrão que define uma interface para criar um objeto, mas permite às classes decidirem qual classe instanciar. O Factory Method permite a uma classe deferir a instanciação para subclasses”.

- Com o padrão Factory Method podemos encapsular o código que cria objetos.

### Definindo os produtos abstratos e concretos que serão utilizados pela factory

```
public abstract class Pessoa {  
    public String nome;  
    public String sexo;  
}  
  
class Homem extends Pessoa {  
    public Homem(String nome) {  
        this.nome = nome;  
    }  
}
```

```

        System.out.println("Olá Senhor " + this.nome);
    }
}

class Mulher extends Pessoa {
    public Mulher(String nome) {
        this.nome = nome;
        System.out.println("Olá Senhora " + this.nome);
    }
}

```

Em tempo de execução não sabemos quem será chamado, ao invés de termos if's e else's no cliente, temos toda a lógica de decisão na factory.

### Implementação do Method Factory

```

class FactoryPessoa {
    public Pessoa getPessoa(String nome, String sexo) {
        if (sexo.equals("M"))
            return new Homem(nome);
        if (sexo.equals("F"))
            return new Mulher(nome);
    }
}

```

### Criando a factory com os dados abaixo

```

public class TesteApp {
    public static void main(String args[]) {
        FactoryPessoa factory = new FactoryPessoa();
        String nome = "Carlos";
        String sexo = "M";
        factory.getPessoa(nome, sexo);
    }
}

```

Toda a "sujeira" (parte da decisão) fica na factory e ela decide o que fazer.

### Abstract Factory

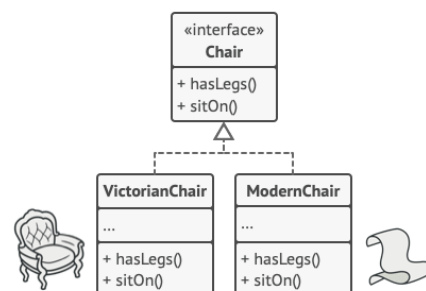
Um padrão de projeto criacional que permite que você produza famílias de objetos relacionados sem ter que especificar suas classes concretas.

Imagine que você está criando um simulador de loja de mobílias. Seu código consiste de classes que representam:

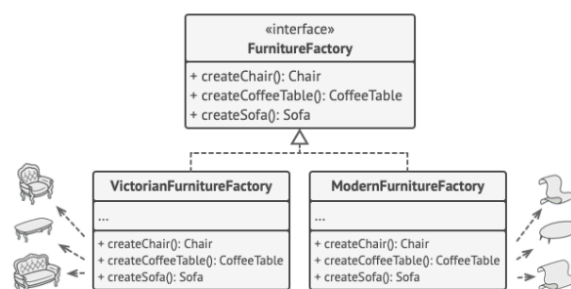
1. Uma família de produtos relacionados, como: Cadeira + Sofá + MesaDeCentro.
2. Várias variantes dessa família. Por exemplo, produtos Cadeira + Sofá + MesaDeCentro estão disponíveis nessas variantes: Moderno, Vitoriano, ArtDeco.

Preciso criar objetos de mobília individuais para que eles combinem com outros objetos da mesma família.

O padrão Abstract Factory sugere declarar explicitamente interfaces para cada produto distinto da família de produtos (ex: cadeira, sofá ou mesa de centro). Então você pode fazer todas as variantes dos produtos seguirem essas interfaces. Ex.: todas as variantes de cadeira podem implementar a interface cadeira.



Depois, é preciso declarar a fábrica abstrata, uma interface com a lista de métodos de criação para os produtos que fazem parte da família de produtos (por exemplo, `criarCadeira`, `criarSofá` e `criarMesaDeCentro`). Esses métodos devem retornar tipos abstratos de produtos representados pelas interfaces que extraímos previamente: Cadeira, Sofá, MesaDeCentro e assim por diante.



*Cada fábrica concreta corresponde a uma variante de produto específica.*

Da fábrica abstrata FurnitureFactory saem a VictorianFurnitureFactory e a ModernFurnitureFactory (fábricas concretas).

Uma fábrica é uma classe que retorna produtos de um tipo em particular. Por exemplo, a classe FábricaMobíliaModerna só pode criar objetos CadeiraModerna, SofáModerno, e MesaDeCentroModerna.

1. **Produtos Abstratos** declaram interfaces para um conjunto de produtos distintos mas relacionados que fazem parte de uma família de produtos.

2. **Produtos Concretos** são várias implementações de produtos abstratos, agrupados por variantes. Cada produto abstrato (cadeira/sofá) deve ser implementado em todas as variantes dadas (Vitoriano/Moderno).
3. A interface **Fábrica Abstrata** declara um conjunto de métodos para criação de cada um dos produtos abstratos.
4. **Fábricas Concretas** implementam métodos de criação fábrica abstratos. Cada fábrica concreta corresponde a uma variante específica de produtos e cria apenas aquelas variantes de produto.